

Mathematical programming and resource management,

J T Buchanan

This paper reviews important aspects of the current state of mathematical programming — the use of optimisation techniques to model and solve constrained optimisation problems. The approach is non-technical, but seeks to give an understanding of the mathematical programming paradigm and its scope, computational issues, and important system features enabling the use and understanding of mathematical models. Within the limits of reported results, examples are given of the approach to large-scale mathematical programming applications in resource management and some conclusions are drawn about the requirements to generate successful applications.

1. Introduction

Mathematical programming, sometimes referred to generically as optimisation, is generally agreed to have emerged from an intensive period of post-war research and development, when Dantzig and others made seminal contributions in three related areas. Firstly, they recognised the opportunities of formulating many types of problem, but especially those of resource and operations management, as constrained optimisation problems. Secondly, they developed mathematical algorithms to solve these problems, and finally they implemented programs for the machine-resident computation involved in applying the algorithms to problems then regarded as of an interesting size - certainly beyond the ability of humans to calculate on a regular basis. Apart from the conventional material, Dantzig [1] describes something of the early history of the subject.

The period since that fruitful start has been one of serious and sustained development in several areas including:

- extensions of the modelling and solution effort into new technical areas (involving nonlinear, combinatorial and stochastic elements),
- the discovery of new types of algorithms to challenge established methods,
- considerable improvements in the underlying algorithms themselves,
- vastly enhanced implementations of the numerical methods,
- an increasing focus on facilities (environments) to allow the human analyst or decision maker a better understanding of the processes of model creation, interpretation and management.

There is an excellent, essentially non-technical, introduction to the power and scope of these techniques [2] and also a complementary technical coverage of the algorithms and implementation issues [3]. Systematic technical coverage of important areas of the mathematical material can be found in Nemhauser et al [4], specifically the sections on linear programming [5], constrained nonlinear programming [6], integer programming [7] and network flows [8]. An overview of mathematical programming systems, describing the power and functionality of software packages can be found in Tomlin and Welch [9].

2. The mathematical programming paradigm

In a mathematical programming approach, the problem under study is formulated using variables and constraints. The variables represent decisions (or their consequences) and the constraints describe relationships among variables. The variables may be continuous (in linear and nonlinear programming) or discrete (as in integer programming), and indeed a model may contain both classes of variable (as in mixed integer programming). Constraints may involve linear or nonlinear elements, and collectively may have significant structural features, e.g. reflecting the basic constraints of a network-like model or reflecting something of the real-world problem, such as near decomposability.

Some classes of constraints, such as simple interval bounds on variables, can be treated implicitly in the numerical workings of the model, and significant features of the model can be exploited by algorithms and/or the software which implements them.

The computational methods used to solve large-scale models typically involve the extensive use of

advanced techniques of numerical analysis, and concepts about sparse data structures and their implementation when using large data sets.

It is a feature of the important numerical methods that they can be used very effectively in an incremental manner, responding for example to some form of perturbation analysis — ‘what if the price of X increases by 10%?’ — or where the algorithm is being used in a context which continually produces a succession of related problems, e.g. in some methods for integer optimisation. Again, where models are being solved on a recurrent basis, e.g. weekly runs of a base model with potentially different data and some relatively minor structural changes, the algorithms and their implementations have properties, whereby the effort of solving an earlier instantiation of the problem may readily be exploited in initiating the solution process for a new version. This can be extremely effective in reducing the computational effort.

While mathematical programming methods are normally thought of as synonymous with optimisation, it is not the case that they are restricted to finding the optimal value of an objective function, subject to a set of constraints. In goal programming, the same techniques are used to find a satisfactory solution, i.e. one which satisfies a set of constraints and simultaneously reaches aspiration levels for one or more objectives (measures of quality or performance). In fact the distinction between objectives and constraints is an artefact. In the implementation of a solver for mathematical programming systems, it is normal for the system to allow the specification of the relationships in the model and for the user to then select a particular relation to act as the driver (or objective) in the search for (optimal) solutions, or for the system to find a feasible solution of the set of relationships specified, without an explicit objective from the user to drive the search.

A model is commonly based on some existing or planned system, and as such is an interpretation of that system. The modelling process generally involves decisions about the inclusion/exclusion of particular features of the “real” system. Elements may be excluded because they are thought to be insignificant for the purposes of the model, or because they cannot be captured within the modelling technology being used. This is true of all modelling activity and not confined to mathematical models. In addition to this problem (really concerned with the validation and verification of the model) there may be alternative methods of representing the same underlying problem. This can be important, especially in models with a significant combinatorial component.

3. The scope of mathematical programming

The range of models available within the scope of mathematical programming is extensive, though it is unified by the notions of variables, constraints, and goals or objectives. The most common model - and also the most compu-

tationally tractable — is that where the variables are all continuous (each typically belonging to some finite or semi-infinite interval) and each constraint involves some linear combination of (a subset of) the problem variables. Such linear programming models are important in practice, have been extensively studied in terms of computational performance, and are an important element in the more complex models identified below [5].

Important special cases of linear programming occur when the problem structure is reflected in a data structure able to be exploited in an algorithmic manner. This exploitation may be a function of known mathematical properties of the special case, e.g. in network models where ‘specialisations’ of the standard algorithms offer computational improvements of several orders of magnitude [8, 10]. In Laithwaite [11], there is an example of such a specialised algorithm and its application. The specialisations often appear very far removed from the base algorithm and certainly are implemented in radically different ways to gain maximum computational advantage. Such models are to be found in problems associated with flows in networks, transportation, allocation and assignment. Many ‘real’ problems tend to have a very large structured component with a few complicating features which break the purity of the model. Again there are special versions of the numerical methods which attempt to exploit what structure there is, as well as coping with the messier, complicating features.

Another example of a special case is where the underlying model has properties, e.g. capturing a multi-period effect on the use of resources, which lead to prominent structure in an otherwise normal linear programming formulation. Again, specialisations of the base algorithms can lead to computational advantage, e.g. enabling models to be tackled that are of a larger-scale than in the unstructured case [5].

As well as being a fundamental problem-solving tool in its own right, the simplex method for linear programming is used as a component in other optimisation algorithms for nonlinear and integer programming. The underlying efficiency of implementations of simplex (or other algorithms for the linear programming problem) is thus key to good computational performance on a whole range of modelling activities.

Second in importance to linear models are those where some or all of the variables are discrete, perhaps taking one of two values (binary variables) or one from a small finite set. The former are often used in modelling to reflect either/or and on/off type decisions, while the latter represent a set of alternatives. An example of the former would be where a decision has to be made about the inclusion or exclusion of a particular component in a mixture, while an example of the latter is in deciding on the location of a facility at one of

several locations. In both models there will often be a further requirement to decide on a quantity, e.g. amount of element, capacity of facility.

Integer programming models are used to represent such problems. In the simplest cases the constraints continue to be linear but some or all of the variables may now be discrete, with the remainder continuous as before. Just as ‘sentences’ can be constructed in formal logical formats expressing alternatives (such as the use of ‘and’, ‘not’, etc), so too can integer programming capture this complexity. However, it is very easy to be faced with a problem where capturing the logic leads to nonlinear combinations of variables and hence to nonlinear integer programming.

Models where the variables remain continuous, but the constraints involve nonlinear combinations of variables, fall in the area of nonlinear programming [6]. Special cases of the general case, for example where the goal is quadratic and the constraints are linear, have been extensively explored with significant developments in a theoretical understanding of the problem, as well as computational procedures for tackling problems of a large scale [12].

4. Computational issues

4.1 Progress in linear programming

In mathematical programming, developments in mathematical understanding, numerical analysis, software and hardware have reinforced one another, leading to continual, impressive improvements in the computational performance of systems from their first implementations in the late 1940s, to a range of quality systems which are available on machines extending from PCs through workstations to supercomputers [13]. Until the mid-1980s the revised simplex algorithm [5] was unchallenged as the tool for large scale linear optimisation. Despite theoretical worst-case behaviour which is exponential, in practice it inevitably behaves as a low-order polynomial algorithm. Indeed, it requires considerable ingenuity to make it behave as badly as it might [14]. This fortunate property is doubly important in that elements of revised simplex are themselves used as a subroutine within algorithms for nonlinear and integer programming.

The ellipsoid algorithm of Khachyan [15], although possessing a low-order polynomial bound on computational complexity, proved very poor in terms of computational efficiency when implemented. Karmarkar’s projective algorithm [16] was the first interior-point method which simultaneously had good theoretical and practical performance. There is also now considerable interest in the comparative computational merits of the two competing methods and work on their integration for mutual benefit. The relative merits of the two approaches are still the

subject of research, and this is described in section 4.3 below.

A recent quasi-historical review of developments in optimisation is to be found in Nemhauser [17]. The enhanced functionality of modelling systems is due to the interaction of new advances in algorithms as well as system improvements (machine speed, compiler quality, better development environments etc). Problems now within the scope of linear programming can have in excess of one million variables and thousands of constraints. Such a model is inevitably sparse, i.e. the matrix representing the constraints will have a very small fraction of nonzero entries, usually less than 1%, often less than 0.1%. It is traditionally believed that the average number of constraints within which one particular variable appears is six. In a model with thousands of constraints, the density is thus very low indeed.

In Nemhauser [17], an analysis is given of the computational performance of the revised simplex algorithm over several generations of machine and optimisation system. In examining the parameters for solving a problem well-known to the community (involving 6711 variables and 4422 explicit constraints) there is not only the expected improvement in time to solution (from 550 min down to 4 min) but also a dramatic reduction in the number of iterations taken (from 302K down to 5K). The combination of better platforms and better solution systems reinforce one another. Again, Nemhauser [17] comments on the scalability of implementations of revised simplex, where the iteration count (and hence solution time) grows linearly with the number of constraints. This has remained true across several generations of the algorithm.

4.2 Integer programming

The same review offers a number of comments on developments in (mixed) integer programming. Again the solution systems have evolved rapidly, though a number of issues are significantly different from the purely linear case. Linear programming methods are very adept at handling perturbations to the initial problem, in data and even in structure, so that reoptimisation on a perturbed problem is very efficient. Experience with integer programming methods has been more mixed, despite their use of underlying linear technology.

Given that (mixed) integer programming problems have a logical (or combinatorial) flavour they are often susceptible to alternative, but logically equivalent, formulations. Experience clearly indicates that a good formulation is important [2, 18], as is an element of pre-processing, such as fixing variables, eliminating redundant constraints, and tightening constraints. These procedures are all applicable to the pure linear case, but it is in the context of integer programming that they may become critical.

There is consistent evidence to support the case that preprocessing reduces solution time (with some impressive examples) and generally gives the solution process a ‘good start’ to the search for optimality by reducing the gap between solutions to the relaxed (linear) problem and the parent (discrete) problem.

A powerful example of the benefits of reformulation in the context of a distribution problem is to be found in Barnhart et al [18]. The original discrete model proved to be insoluble practically, no solution appearing within lengthy computer runs on powerful computers. A new equivalent model, with a reduced number of binary variables and a ‘tighter’ formulation solved to optimality in 10 minutes on a workstation. A recent survey of (mixed) integer programming systems is to be found in Saltzman [19].

4.3 Interior point versus edge walking

With respect to the interior point versus simplex competition, Lustig et al [20] give a review of the computational state of the art on interior (projection) methods for linear optimisation. The authors note that methods of this type have been around for nonlinear problems since the mid-1950s, and were seriously developed (in analytic and computational terms) in the late 1960s [6, 12]. Only with the work of Karmarkar has interest in interior-point methods been raised in the purely linear context. Very large problems are now routinely solved by both interiorpoint and revised-simplex methods. The interior-point methods are certainly competitive, if not as full-featured as revised simplex, for example lacking ‘warm start’ or reoptimisation capabilities, and being generally weaker at detecting unfeasible and unbounded problems.

A complementary paper [21] indicates that Lustig et al [20] possibly overstate some results. In the situation where codes are developing rapidly, and leapfrogging one another in ability, meaningful comparison is difficult. Regardless of the eventual outcome of this interesting competition, Bixby [21] notes running time improvements for revised simplex of a factor of 40 over three generations of system. This easily exceeds pure processing improvement and confirms the conclusions of Nemhauser [17].

4.4 Hybrid approaches

As with many solution technologies, problems of scale and complexity yield more readily, if at all, to an approach which is more sophisticated than just ‘formulate and solve’. Such a naive approach will work — even for problems of some scale — but it is the case that problems at the edge of tractability will need a more subtle and inventive method. Examples of this practice are to be found in both continuous and discrete modelling applications.

In McKinnon and Buchanan [22] pre- and postprocessing is used on an intractable large virtual model to present to a solver a still large, but computationally tractable, reduced model. The preprocessing is ‘smart’, interrogating a declarative description of the model [23] to determine which variables and constraints need to be incorporated into the reduced model for direct solution and which may be solved a posteriori by substitution after optimisation.

Hybrid methods, such as Anbil et al [24], are commonly used for models with a significant combinatorial component. This combination of heuristic and optimisation techniques addresses an economically significant manpower scheduling problem. The virtual problem is again too large to solve, and preprocessing is used to reduce the search space for the optimisation routines.

As well as pre- and post-processing, other devices are commonly used within the optimisation routines to enhance computational performance. In large models, for example, implementations of revised simplex may work iteratively with subsets of the total problem data, eventually ensuring optimality, but not by working simultaneously with global data sets.

All of this experience indicates that advanced use of mathematical programming methods requires that the solution technology is not seen as a black box, but that skilled analysts must apply imagination and experience to solve problems at the very edge of tractability. Sometimes, however, the facilities offered within vendors’ products can limit the extent to which this can be achieved.

4.5 Parallel computing

Finally, the scope for the use of parallel computing in mathematical programming (and associated areas such as large-scale linear algebra) has been extensively studied [13, 25]. Most of the results here concentrate on fundamental aspects of the underlying linear algebra (e.g. matrix factorisation) or issues associated with nonlinear problems. Certainly there has been little success with attempts to parallelise the revised simplex method. The branch-and-bound strategy associated with the solution of integer programming problems is an obvious candidate for parallel computation (certainly of a coarse grained nature [26]), but experience so far indicates that optimisation methods share with other areas the common questions of how to map algorithms to platforms, and how to control the communication/computation balance for maximum benefit.

5. Modelling languages and user interfaces

5.1 The modelling environment

The computer environment, within which mathematical programming is pursued, has changed over the years reflecting the changes in the underlying computational paradigm (batch, time-sharing, workstation), developments in general user interfaces, and developments in the facilities given to users to specify and manage their mathematical models.

Regardless of pre- and post-processing, every model has to have a particular mathematical structure (usually matrix-based) described and populated with data. Early methods for doing this involved working directly with the structure and data itself, e.g. specifying the non-zero entries of the matrix in a column oriented manner [27]. This is the antithesis of user-friendly, working with highly formatted data in an unintuitive way, well removed from any natural description of the underlying model. A more natural description is algebraic, reflecting through syntax the components and constructs to be found in algebraic expressions. Such languages have evolved over the last 20 years. Issues associated with algebraic modelling languages are explored in Maturana [28], evaluating the strengths and weaknesses of major existing examples, though without systematic comparison. The user expectations expressed reflect those of Geoffrion [29] in his 'structured modelling' work, where the aim of the environment is:

- to nurture the entire modelling life cycle,
- to be hospitable to decision and policy makers, not just modelling professionals,
- to facilitate the maintenance and evolution of models and systems,
- to enable all co-users to converse in a paradigmneutral language of model definition,
- to facilitate good management of key resources (including data, models, solvers and knowledge derived from them).

In Kuip [30] a survey of algebraic modelling languages is given. These have common features (allowing the description of model elements in algebraic terms) but differ in terms of the language features used. Kuip attempts to evaluate these features on the basis of their contribution to the human aspects of understanding, maintaining and verifying model descriptions.

Finally, the declarative modelling languages (such as CHIP, PROLOG III and Ilog Solver) have extended in functionality from being purely logic-based to including elements of numerical problem-solving [31]. In these systems the logical basis is naturally dominant and

most highly developed, and the numerical processing capabilities are, by comparison, very limited. As such, their applicability is currently limited to models whose constituent parts reflect this balance.

5.2 Graphical interaction

The general interest in graphical tools has extended to mathematical modelling systems. In Colland and Pasquier-Boltuck [32] the user is allowed to describe classical linear problems in terms of graphical objects which are then connected to form a model. A demonstration system, based on the Macintosh, is described. As with other examples of graphically-based languages, there are unresolved issues about the richness of the syntax available to the user and the scalability of the technology when faced with having to describe large models.

A different type of user interaction, again using a graphical base is to be found in Buchanan and McKinnon [33]. The aim of this system is to allow the user to explore the solution space corresponding to a dynamically definable model. In one operational mode, the user selects a number of putative control variables and a number of performance variables from the model confronting him. Through mouse interaction on analogue instruments the user can change the values of control variables and instantaneously see the effects on the performance variables. The user can specify target or constraint intervals for variables and informally evaluate the quality of solutions satisfying these relationships. Again, the user can request the system to find a solution (if possible) in an area of the search space with particular variable values or ranges; indeed, the user can request the system to optimise on a chosen variable. In all cases, the user is then free to explore from the point in question, and further free to repartition the problem with different control/ performance variables. The fluency of the system relies on a number of advanced mathematical developments which can process the effects of user manipulation sufficiently rapidly to give the impression of 'animating' the model.

6. Mathematical programming and resource management problems

Since the very earliest days, problems drawn from the domains of planning, scheduling, resource management and operations management have been an important component of the applications to which mathematical programming systems have been devoted. A recent compendium with large-scale applications, as well as more mathematical papers, is to be found in Ciriani and Leachman [34].

In this section examples are drawn from three important application areas or problem classes. As is generally the case in the reporting of (attempted) applications, the

literature here is patchy, since it is seldom possible to give anything other than a broad indication of the scale and complexity of the underlying problem and the important connections between problem and model.

6.1 Planning and scheduling

The context for modelling, including mathematical programming, is changing [35]. What is of concern now is the ability of managers to acquire and organise production data, to which added value is provided by, among other things, a modelling system. In other words, decision support systems have emerged as a generic concept with real applications and real pay-offs. The concern in this paper is to present models and solutions that have actually been applied or show reasonable promise for practical application. Material is wide ranging, working through some very simple models, decomposition methods (adding/exploiting structure in the modelling and solution processes) and continuing with studies in three main areas of manufacturing.

The first analysis is a study of process manufacturing models founded on very simple models (e.g. travelling salesman) where again there is the prospect for a hybrid approach which embeds an optimisation problem (here of a very simple type) in a heuristic shell. The combination of heuristic and optimiser begins to allow the modeller to address some of the complexities of the real problem. Building as it does on very simple components (for which efficient computational procedures exist), this approach will be representative of that found on models at the very edge of computational tractability. Indeed such an approach is not confined to mathematical methods, but can be found in advanced applications of more heuristic techniques.

A second analysis demonstrates the power of resourcedirected decomposition methods to exploit in an algorithm manner the types of problem structure often reflected in a model structure. For example, the natural distribution of a problem, be it temporal, physical or logical, can be captured in a mathematical formulation which is susceptible to decomposition, and hence solution, by algorithmic strategies exploiting this model feature. Again this general strategy can be found in other problemsolving approaches. Shapiro [35] also notes the match between these methods (now well established — for the mathematical programming case — in theory and implementation) and coarse-grained computing platforms.

Finally, there is an examination of the linkages between production and other company systems, reflecting growing concern (in the domain) of the interlinking of production, information and other management systems.

6.2 Mathematical programming and management information systems

An interesting example of the scope of mathematical programming models and the benefit of linking them with conventional management systems can be found in Leachman [36]. Here, several uses are made of the same model. The domain (semiconductor manufacture) is characterised by its high capital costs and the nature of its discrete part-product flow. The systems described are a combination of linear programming and material requirements planning (MRP). As with many conventional manufacturing management systems, the global planning system is capacity oriented and spans several production facilities. It is capable of providing an “availability schedule” for use with customer delivery and order enquiries, but is not itself decision-oriented. The linear optimisation element reflects the constraints and requirements of production, inventory and marketing. If formulated naively as a single model the raw problem would have some 500K constraints, and hence be computationally demanding or even intractable. As observed above, such a predicament usually dictates the need for some heuristic scheme, to be used in association with an optimiser. In this case the heuristic used is that of decomposition. Potential areas for decomposition are at separate functions within the overall model (e.g. requirements planning and capacitated loading calculations), or at different parts of the manufacturing network (represented within the one virtual model). As with many forms of decomposition, the procedure may involve loss of optimality, and needs to be undertaken with care. In this case the actual decomposition is heavily informed by domain considerations.

Another example of a case where classical dataoriented manufacturing methods are combined with, or subsumed by, decision-oriented models is that reported by Escudero and Kamesan [37]. Here, a stochastic programming model is proposed for MRP under uncertainty and with non-stationarity in product demand. The system objective is to develop effective computational mechanisms which exploit problem structure, in this case the fact that the required model is multi-product, multilevel, and multi-period. Although the computational experience is incomplete there are indications that the approach may be worthwhile, especially since it has clear potential for using parallel processing.

A suite or hierarchy of three computer-resident decision-support systems is described in Gascon et al [38], covering the inter-related areas of forecasting, planning and scheduling. The items being manufactured are household products. The forecasting model covers a four-month rolling horizon, computed on a monthly basis using regression on historical data. The output from this model is the input to the planning and scheduling systems. The

production planning system, incorporating an optimisation model to minimise production costs, delivers a four-month production plan for each product, incorporating decisions on strategic inventory control, contract manufacture and distribution. Finally a scheduling system supports the scheduling of production cycles on individual packing lines at each plant. In this instance, the mathematical programming contribution is mainly in the middle tier of the hierarchy.

6.3 Combinatorial models

A classic application domain for linear programming — that of refinery modelling — is described in Ballantijn [39]. The requirement is for the daily scheduling of production plant, processing raw material and other products through the usual range of refinery activities. In this case, Ballantijn demonstrates that a naive linear model leads to an unacceptable number of switches between operational modes. The underlying problem is multi-period and the naive model scatters operational modes over time, a poor solution from the pragmatic or operational point of view. In an enhanced integer model, the extended modelling technology is used to constrain further the description of acceptable solutions, and this has the desired effect.

In Hoitomt et al [40] a model is proposed for the scheduling of a job shop. A form of decomposition is used within a hybrid computational model to produce solutions which are of good quality when measured against known bounds for performance. One of the techniques being used (Lagrangean relaxation) is a common device to allow for the decomposition of a large model into several interrelated sub-models. An iterative approach is then used to solve such parameterised sub-models and thus devise a solution to the global problem. The authors claim favourable computational results when compared to a knowledge-based system currently in operation within a site under study.

A large system which has attracted considerable attention is described in Anbil et al [24], where the problem is to determine flight crew arrangements for a large set of aircraft, effectively servicing mainland USA. The labour being managed is very expensive (annual labour costs are 1.3 billion dollars) and use of labour is constrained by a variety of factors, including mandatory requirements determined by regulatory agencies. Again, any naive model would be overwhelmingly large and incapable of solution, so a combination of heuristic and optimisation techniques are used. Heuristic preprocessing massively reduces the search space passed to the optimisation algorithms, and this combination leads to claimed annual savings of some 25 million dollars.

7. Conclusions

In terms of solving large-scale problems, the main issues and difficulties associated with the mathematical programming approach include:

- increasing computational power through better algorithms, implementations and platforms,
- improving ease of use through more expressive modelling facilities and fluent interfaces,
- a need to detect and exploit problem structure,
- a recurring need (related to the point above) to embed the underlying optimisation technology in a heuristic context, with pre- or post-processing of the model, or using the optimiser in a heuristic search,
- a requirement to integrate models with existing systems.

It should be noted that these points could equally well be made of other problem-solving approaches. Some of the issues are briefly explored below.

Mathematical programming offers considerable expressive and computational power to a model builder. A combination of technical advances and progress within the subject itself has given users desk-top power which was only available on large and powerful mainframe computers a few years ago. Expected improvements in computational performance will further enhance this accessibility, but in many ways this is less important than other potential developments.

Ease of use — by specialists during development and users with a fielded system — continues to be critical to the uptake of modelling projects. As with any computerresident system, the process of moving from the establishment of user needs to a satisfied client is far from straightforward, and a more productive computer environment in which to prototype is always welcome. The expressiveness of modelling languages and the ability of non-specialists to move close to the modelling process, in the sense of understanding (and hence validating) the models being offered, are both important areas. Developments using symbolic, non-procedural approaches are demonstrating very considerable potential in this regard.

It will always be easy to find real problems (of a combinatorial nature) which tax the computational power of any machine, and there will be a persistent need for models which are developed with an eye to effective computation. Integer programming models are an obvious area where a “good” formulation, derived manually or with intelligent support, can yield benefits in reducing the computation of acceptable or optimal solutions.

Regardless of the underlying technology, it is becoming increasingly common for models to be integrated with

conventional (legacy) systems, adding value to the information stored in these systems. This is rather unglamorous, compared to leading-edge research and development, but necessary if mathematical programming - indeed any modelling discipline — is to flourish further.

A recurrent theme in the earlier sections is the need to use hybrid heuristic/optimisation methods to address (combinatorial and continuous) problems of very large scale. Regardless of improvements in computing power, this need is set to persist - greater computer power just raises the stakes in terms of model size and user expectations. The hybrid area is something of a mixture, with a few very well established algorithmic strategies and some rather more ad hoc approaches. If the mainstream elements of mathematical programming are stable, or even if there is yet another shock to the classical methods as in Karmarkar, the hybrid area looks like being one of the most interesting to pursue, since, again, the expectations of users will always exceed the ability of systems to deliver and modellers will continually explore the hybrid route to expand the frontiers of the tractable.

Currently the heuristic element of the hybrid methods has been most influenced by a mathematical view of the model, but there is obvious benefit in exploring some of the knowledge-base systems' ideas on problem-solving strategies and control of search. Unfortunately these two communities have not yet learnt much from one another — indeed no serious comparative study of the technologies has been undertaken, though several elements of folklore exist.

Finally, there is scope for the inventive use of mathematical programming methods in a non- classical mode — for example in exploring solution spaces and even exploring spaces of possible models. In this context, navigation, rather than the mathematics, is the challenge.

References

- 1 Dantzig G B: 'Linear programming and extensions', Princeton UP (1963).
- 2 Williams H P: 'Model building in mathematical programming', Wiley, 3rd edition (1990).
- 3 Williams H P: 'Model solving in mathematical programming', Wiley (1993).
- 4 Nemhauser G L, Rinnooy Kan A H G and Todd M J (Eds): 'Optimisation', North-Holland (1989).
- 5 Goldfarb D and Todd M J: 'Linear programming', in Nemhauser G L et al (Eds): 'Optimisation', pp 73—170, North-Holland (1989).
- 6 Gill P E, Murray W, Saunders M A and Wright M H: 'Constrained nonlinear programming', in Neumauser G L et al (Eds): 'Optimisation', pp 211—370, North-Holland (1989).
- 9 Tomlin J A and Welch J S: 'Mathematical programming systems', in Coffman E G et al (Eds): 'Computing', pp 561—601, North-Holland (1992).
- 10 Glover F, Klingman D and Philips N V: 'Network models in optimisation and their applications in practice', Wiley Interscience (1992).
- 11 Laithwaite B: 'Work allocation challenges and solutions in a large-scale work management environment', BT Technol J, 13, No 1, pp 46—54 (January 1997).
- 12 Fletcher R: 'Practical methods of optimisation', Wiley (1987).
- 13 Levkovitz R and Mitra G: 'Solution of large-scale linear programs: a review of hardware, software and algorithms issues', in Ciriani T A and Leachman R C (Eds): 'Optimisation in industry: mathematical programming and modelling techniques in practice', pp 139—171, Wiley (1993).
- 14 Klee V and Minty G J: 'How good is the simplex algorithm?', in Shisha O (Ed): 'Inequalities-III', Academic Press (1972).
- 15 Khachyan L G: 'A polynomial algorithm for linear programming', Doklady Akad Nauk, USSR, 244, No 5, pp 1093—96 (translated in Soviet Mathematics Doklady, 20, pp 191—194) (1979).
- 16 Karmarkar N: 'A new polynomial time algorithm for linear programming', Combinatorica, 4, pp 373—394 (1984).
- 17 Nemhauser G L: 'The age of optimisation: solving large-scale real-world problems', Op Res, 42, No 1, pp 5—13 (January/February 1994).
- 18 Barnhart C, Johnson E L, Nemhauser G L, Sigismondi G and Vance P: 'Formulating a mixed integer programming problem to improve solvability', Opns Res, 41, pp 1013—1019 (1993).
- 19 Saltzman M J: 'Survey: mixed integer programming', OR/MS Today, pp 42—51 (April 1994).
- 20 Lustig I J, Marsten R E and Shanno D F: 'Interior point methods for linear programming: computational state of the art', ORSA J on Comp, 6, No 1, pp 1—14 (Winter 1994).
- 21 Bixby R E: 'Progress in linear programming', ORSA J on Comp, 6, No 1, pp 15—22 (Winter 1994).
- 22 McKinnon K I M and Buchanan J T: 'The short term scheduling of a hydro-thermal electricity generating system', in Osiadacz A J (Ed): 'Proc IMA conf on large scale optimisation and simulation', OUP (1988).
- 23 Buchanan J T and McKinnon K I M: 'An example of a declarative approach to model creation', in Stein S W (Ed): 'Algorithms and model formulations in mathematical programming', NATO ASI Series, Springer-Verlag (1989).
- 24 Anbil R, Barnhart C, Hatay L, Johnson E L and Ramakrishnan V S: 'Crew-pairing optimisation at American Airlines Decision Technologies', in Ciriani T A and Leachman R C (Eds): 'Optimisation in industry: mathematical programming and modelling techniques in practice', pp 31—36, Wiley (1993).
- 25 Bertsekas D P and Tsitiklis N J: 'Parallel and distributed computation', Prentice-Hall (1989).
- 26 Brown R W, Shapir J F and Waterman P J: 'Parallel computing for production scheduling', Manuf Sys, 6, pp 56—64 (1988).
- 27 Murtagh B A: 'Advanced linear programming', McGraw-Hill (1981).
- 28 Maturana S V: 'Issues in the design of modelling languages for mathematical programming', EJOR, 27, No 2, pp 243—261 (January 1994).
- 29 Geoffrion A: 'Introduction to structured modelling', Mgt Sci, 33, pp 547—588 (1987).
- 30 Kuip C A C: 'Algebraic languages for mathematical programming', EJOR, 67, pp 25—51 (1993).

- 31 Lever J, Wallace M and Richards B: 'Constraint logic programming for scheduling and planning', *BT Technol J*, 13, No 1, pp 73—80 (January 1995).
- 32 Collaud G and Pasquier-Boltuck J: 'gLPS: a graphical tool for the definition and manipulation of linear problems', *EJOR*, 72, No 2, pp 277—286 (1994).
- 33 Buchanan J T and McKinnon K I M: 'An animated interactive modelling system for decision support', *EJOR*, 54, pp 306—317 (1991).
- 34 Ciriani T A and Leachman R C: 'Optimisation in industry: mathematical programming and modelling techniques in practice', Wiley (1993).
- 35 Shapiro J F: 'Mathematical programming models and methods for production planning and scheduling', in Graves S C et al (Eds): 'Logics of production and inventory', pp 371—443, North-Holland (1993).
- 36 Leachman R C: 'Modelling techniques for automation production', in Ciriani T A and Leachman R C (Eds): 'Optimisation in industry: mathematical programming and modelling techniques in practice', pp 1—30, Wiley (1993).
- 37 Escudero L F and Kamesam P V: 'MVP modelling via scenarios', in Ciriani T A and Leachman R C (Eds): 'Optimisation in industry: mathematical programming and modelling techniques in practice', pp 101—111, Wiley (1993).
- 38 Gascon A, Leachman R C and DeGuia A A: 'Optimal planning and control of consumer products packaging lines', in Ciriani T A and Leachman R C (Eds): 'Optimisation in industry: mathematical programming and modelling techniques in practice', pp 173—189, Wiley (1993).
- 39 Ballantijn K: 'Optimisation in refinery scheduling: modelling and solution', in Ciriani T A and Leachman R C (Eds): 'Optimisation in industry: mathematical programming and modelling techniques in practice', pp 191—199, Wiley (1993).
- 40 Hoitomt D J, Luh P B and Pattipati K R: 'A practical aproach to job-shop scheduling problems', *IEEE Trans on Robotics and Automation*, 9, No 1, pp 1—14 (February 1993).



J T (Iain) Buchanan is Professor of Computer Science and Head of Department at the University of Strathclyde.

He has research interests in knowledge-based and mathematical modelling, with an emphasis on applications involving resource and operations management. Projects in this area have received substantial support from Research Council, DTI and industrial sources including BT.